Relatório 8 — Verificador de senha com Flutter

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Desenvolvimento remoto no Codex Cloud; sem declaração de participação presencial.

Objetivo e fundamentos

Criar uma interface Flutter que analise localmente uma senha e atualize sua classificação. Validação de dados aplica regras explícitas às entradas. Uma senha longa e única tende a resistir melhor a tentativas de adivinhação, mas uma regra de tamanho e classes de caracteres não mede sozinha a segurança efetiva.

Flutter organiza a tela em widgets. MaterialApp define o tema; Scaffold estrutura a página; TextField oculta o texto e usa onChanged; setState atualiza um Text com rótulo e cor. O resultado é comunicado por texto, além da cor, e Semantics com liveRegion favorece a indicação da mudança a tecnologias assistivas.

Critérios adotados

O roteiro não define limites numéricos. Nesta implementação, Forte exige pelo menos 12 caracteres e quatro classes; Média exige pelo menos 8 caracteres e duas classes, desde que não seja Forte; os demais casos são Fraca. As classes são minúsculas ASCII, maiúsculas ASCII, dígitos e símbolos diferentes de espaço. A contagem usa runes (pontos de código Unicode), não unidades UTF-16. Campo vazio solicita digitação, sem classificação.

A heurística é didática: não detecta padrões previsíveis, vazamentos, reutilização ou entropia; letras acentuadas entram na classe de símbolos desta regra simplificada. Não há armazenamento nem logs do texto digitado. Usar apenas entradas fictícias nos testes.

Lógica e código integral

```
if (tamanho >= 12 && classes == 4) return 'Senha Forte';
if (tamanho >= 8 && classes >= 2) return 'Senha Média';
return 'Senha Fraca';
```

Código integral: flutter/relatorio08-senhas/lib/main.dart. A função classificarSenha é independente da tela. O callback onChanged calcula o resultado e setState atualiza a interface; o campo vazio restaura a instrução inicial.

Testes e resultados reais

Há testes preparados para tamanho 7/8/11/12, ausência de diversidade, espaço sem pontuação, três classificações e limpeza do campo. Não executados nesta versão: o SDK Flutter depende de storage.googleapis.com, bloqueado pelo proxy (HTTP 403). Nenhuma execução ou captura Flutter foi confirmada. Não se atribui aprovação a testes apenas escritos.

Após liberar o SDK: executar flutter pub get, flutter analyze e flutter test. Compilar para web e abrir no navegador. Usar três entradas exclusivamente fictícias que atendam às regras e registrar cada rótulo. Verificar também retorno à instrução inicial ao apagar o campo. Não registrar senhas reais em logs, capturas ou relatórios.

Conclusão e aplicação prática

A implementação explicita as regras para permitir testes reproduzíveis. Em sistemas reais, um indicador deve orientar sem prometer segurança absoluta. São necessários senhas longas e exclusivas, proteção de transporte, armazenamento no servidor com algoritmo apropriado de derivação de senha e autenticação multifator. A atividade ainda depende de execução e evidências antes da entrega final.

Evidências de execução

EVIDÊNCIA PENDENTE — EXECUÇÃO NECESSÁRIA

Nenhuma imagem artificial ou captura de outro aplicativo foi usada. Esta seção será substituída por capturas autênticas, com legenda, assim que o projeto puder ser executado. Status: parcialmente concluído.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 8. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 8. Material didático fornecido.

FLUTTER. Flutter documentation. [S. l.], [s. d.]. Disponível em: https://docs.flutter.dev/. Consulta on-line pendente de liberação de rede.

DART. Dart overview. [S. l.], [s. d.]. Disponível em: https://dart.dev/overview. Consulta on-line pendente de liberação de rede.